



Here are the comprehensive class revision notes for the lab session on arrays, focusing on three main problems: trapping rainwater, spiral matrix, and next permutation.

Class Revision Notes

1. Trapping Rainwater

Problem Description

You are given an array representing the height of buildings. The task is to calculate the amount of rainwater that can be trapped between these buildings when it rains.

Concept Explanation

- **Visualization:** Picture the array as a series of buildings of varying heights on a 2D plane. Imagine rain accumulating between these buildings.
- **Unit Measurement:** Calculate the water in terms of "unit squares".

Steps to Solve

1. **Calculate left max (Lmax) and right max (Rmax) arrays:**

- $L_{max}[i]$: The maximum height encountered from the start to the current building i .
- $R_{max}[i]$: The maximum height encountered from the current building i to the end.

2. **Water Level:** For each building, the trapped water can be determined by

$$\min(L_{max}[i], R_{max}[i]) - \text{height}[i] .$$

3. **Edge Cases:** Handle when the height is greater than or equal to L_{max} or R_{max} .

Optimization

- Utilize forward and backward passes to calculate Lmax and Rmax in linear time, reducing the time complexity to $O(N)$ **【6:6+source】**.



Problem Description

Given a square matrix, you are required to print the boundary elements of the matrix in a spiral order, moving clockwise.

Concept Explanation

- **Spiral Order:** Visualize moving along the edges of a matrix, stepping inward once a full boundary is traced.
- **Matrix Reduction:** After printing each complete outer boundary, focus on the submatrix formed by excluding the outer elements `matrix[6:3+source][6:11+source]`.

Steps to Solve

1. **Initialize Pointers:** Set initial boundary pointers for top, bottom, left, and right edges.
2. **Traverse and Print:** Print elements from top left to right, top to bottom, right to left (bottom row), and bottom to the top (left column).
3. **Iterate Inward:** Adjust pointers inward after completing a full boundary traversal, checking for valid boundaries (i.e., `while top <= bottom and left <= right`).
4. **Handling Edge Cases:** Ensure all edge cases are covered for even-sized matrices.

3. Next Permutation

Problem Description

Rearrange numbers into the numerically next greater permutation. If no such arrangement is possible, reorder in the lowest possible order (i.e., ascending).

Concept Explanation

- **Permutation Theory:** A sequence is in its final permutation when sorted in descending order.
- **Swapping Elements:** This allows you to get to the next lexicographical permutation `matrix[6:10+source][6:17+source]`.

**1. Identify a Breakpoint:**

- Traverse from right to left to find the first pair where `arr[i] < arr[i+1]` .

2. Determine Swap Partner:

- From the end of the list, find the first element that is larger than `arr[i]` . Swap them.

3. Reverse the Suffix:

- This step ensures getting the next smallest permutation. Reverse elements from `i+1` to the end.

4. Edge Cases: If no breakpoint is found (i.e., the array is in descending order), reverse the array to get the smallest order .

These notes cover the key concepts, steps, and optimizations discussed during the live class with analogies and implementations that can help you understand and solve these problems efficiently. Make sure to try implementing these approaches on your own for a deeper understanding.